



BabL42

Technikai dokumentáció

iOS · Android · Flutter · PHP 8 · MySQL · WebSocket · APNs · FCM · LiveKit

v1.2.0 · 2026. június

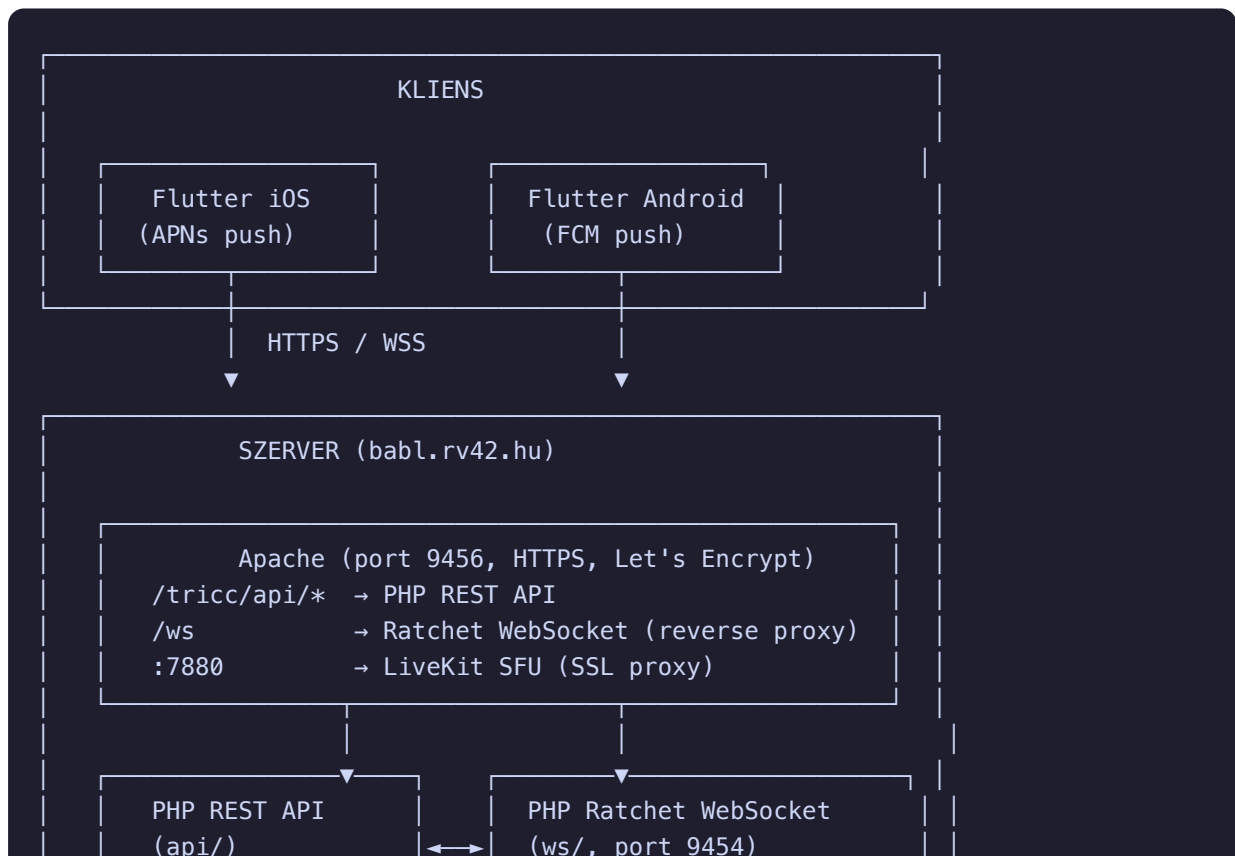
BabL42 — Technikai dokumentáció

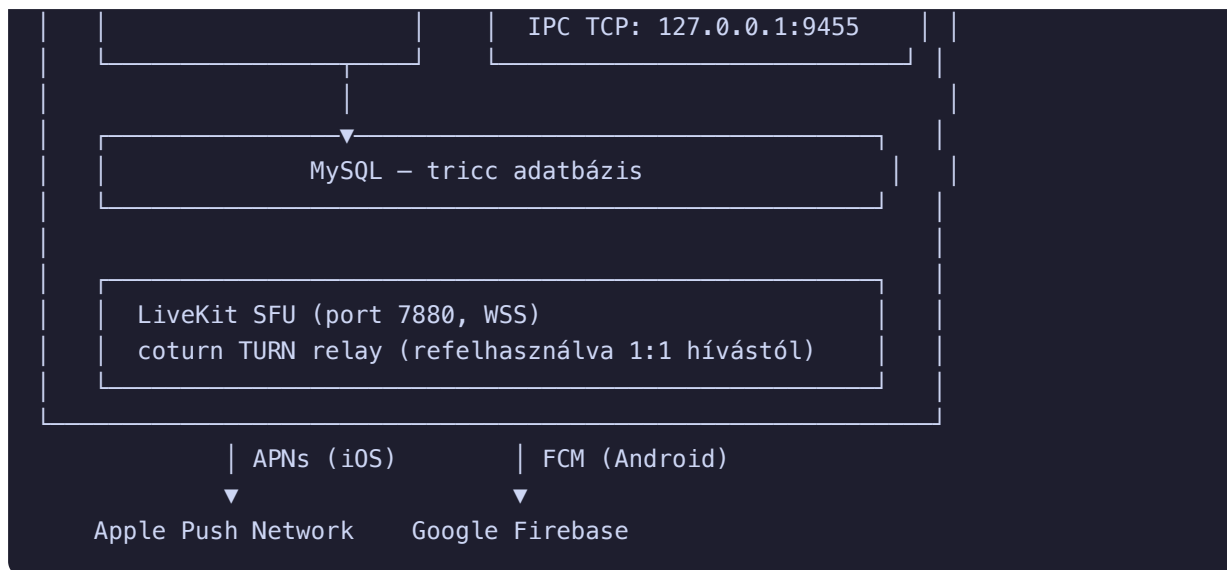
Meghívásos, zárt körű csevegő és hanghívó alkalmazás teljes technikai leírása: kliens (iOS + Android), szerver (REST API + WebSocket + LiveKit SFU), adatbázis, push értesítés, platform channel-ek.

1. Rendszerarchitektúra

1.1 Áttekintés

A BabL42 egy kétszintű (client–server) rendszer. A kliens Flutter alkalmazás iOS-en és Androidon fut; a szerver egy önálló Linux gépen üzemel Apache webserverral.





1.2 Portok és URL-ek

Szolgáltatás	Port	Protokoll	URL
REST API	9456	HTTPS	https://babl.rv42.hu:9456/tricc/api
WebSocket	9456/ws	WSS	wss://babl.rv42.hu:9456/ws
Ratchet (belső)	9454	WS	ws://127.0.0.1:9454
REST→WS IPC	9455	TCP	127.0.0.1:9455
LiveKit SFU	7880	WSS	wss://babl.rv42.hu:7880
coturn TURN	3478	UDP/TCP	babl.rv42.hu:3478

A szerver Let's Encrypt TLS tanúsítványt használ (babl.rv42.hu domain). Standard HTTPS/WSS kapcsolat, nincs szükség cert bypass-ra.

1.3 Mappa-struktúra

```

05_tricc/
├── app/                ← Flutter kliens (iOS + Android)
│   └── lib/
│       └── main.dart

```

```

├── app_theme.dart
├── models/
├── screens/
│   ├── group_call_screen.dart    ← Csoportos hívás képernyő
│   └── video_player_screen.dart  ← Teljes képernyős videólejátszó
├── services/
│   ├── group_call_service.dart   ← LiveKit singleton ChangeNotifier
│   └── call_service.dart         ← 1:1 WebRTC hívás
├── widgets/
│   └── group_call_bar.dart       ← Pip bar (MaterialApp.builder-ben)
├── ios/
│   └── Runner/AppDelegate.swift  ← APNs + proximity platform channel
├── android/
│   └── app/src/main/kotlin/.../MainActivity.kt ← FCM + proximity channel
├── api/                          ← PHP REST API
│   └── src/
│       ├── Controllers/
│       │   └── CallController.php    ← LiveKit token + call/notify
│       ├── APNs.php
│       ├── Auth.php
│       └── DB.php
├── ws/                          ← PHP Ratchet WebSocket szerver
│   └── src/ChatServer.php
├── db/                          ← Adatbázis séma és migrációk
│   └── schema.sql
├── admin/                       ← Web alapú admin felület
│   └── calls.php                 ← Aktív LiveKit hívások oldal
└── docs/                       ← Dokumentáció és tervek

```

2. Adatbázis

2.1 Séma áttekintés

Az adatbázis neve `tricc`, karakterkészlete `utf8mb4`. Összesen 8 tábla.



push_tokens

2.2 Táblák részletesen

users

Mező	Típus	Leírás
id	INT PK AUTO	Felhasználó azonosítója
name	VARCHAR(100)	Megjelenített név
email	VARCHAR(150) UNIQUE	Bejelentkezési email
password	VARCHAR(255)	bcrypt hash
avatar_url	VARCHAR(500)	Profilkép URL
invite_code	VARCHAR(32)	Felhasznált meghívó kód
is_admin	TINYINT(1)	Admin jogosultság
is_active	TINYINT(1)	Aktív-e a fiók
created_at	DATETIME	Regisztráció dátuma

invite_codes

Mező	Típus	Leírás
id	INT PK AUTO	Azonosító
code	VARCHAR(32) UNIQUE	Meghívó kód (pl. <code>TRICC-ADMIN-0001</code>)
created_by	INT FK→users	Ki generálta
used_by	INT FK→users	Ki használta fel (NULL = szabad)
used_at	DATETIME	Felhasználás ideje
expires_at	DATETIME	Lejárat

push_tokens

Mező	Típus	Leírás
user_id	INT PK FK→users	Felhasználó (1 sor/user)
token	VARCHAR(200)	APNs vagy FCM token
platform	VARCHAR(10)	ios vagy android
updated_at	DATETIME	Utolsó frissítés

rooms

Mező	Típus	Leírás
id	INT PK AUTO	Szoba azonosítója
name	VARCHAR(150)	Szoba neve (group)
type	ENUM('direct','group')	Szobatípus
created_by	INT FK→users	Létrehozó
pinned_message_id	INT	Kitűzött üzenet (NULL = nincs)
delete_requested_by	INT FK→users	Ki kért törlést (NULL = nincs)

room_members

Mező	Típus	Leírás
room_id	INT PK FK→rooms	Szoba
user_id	INT PK FK→users	Tag
role	ENUM('admin','member')	Szerepkör
joined_at	DATETIME	Csatlakozás ideje
last_read_at	DATETIME	Utolsó olvasás (unread count alapja)
hidden_at	DATETIME	Elrejtés ideje (NULL = látható)

<code>is_muted</code>	TINYINT(1)	Némított-e a szoba
-----------------------	------------	--------------------

messages

Mező	Típus	Leírás
<code>id</code>	BIGINT PK AUTO	Üzenet azonosítója
<code>room_id</code>	INT FK→rooms	Szoba
<code>sender_id</code>	INT FK→users	Küldő
<code>type</code>	ENUM('text', 'image', 'file', 'link', 'video', 'system')	Típus
<code>content</code>	TEXT	Szöveg tartalom
<code>is_edited</code>	TINYINT(1)	Szerkesztett-e
<code>file_url</code>	VARCHAR(500)	Fájl/kép/videó URL
<code>file_name</code>	VARCHAR(255)	Eredeti fájlnev
<code>file_size</code>	BIGINT	Fájlméret bájtban
<code>reply_to_id</code>	BIGINT	Idézett üzenet ID
<code>reply_to_content</code>	VARCHAR(200)	Idézett tartalom (cache)
<code>reply_to_user_name</code>	VARCHAR(100)	Idézett küldő neve (cache)
<code>created_at</code>	DATETIME(3)	Időbélyeg milliszekundum pontossággal

Az `INDEX idx_room_time (room_id, created_at)` index biztosítja az üzenetek hatékony lapozását.

message_reactions

Mező	Típus	Leírás
<code>id</code>	BIGINT PK AUTO	Azonosító
<code>message_id</code>	BIGINT FK→messages	Üzenet
<code>user_id</code>	INT FK→users	Felhasználó
<code>emoji</code>	VARCHAR(10)	Emoji (pl. 👍)
—	UNIQUE(message_id, user_id, emoji)	Egy user egyszer reagálhat adott emoji-val

message_deliveries

Mező	Típus	Leírás
<code>message_id</code>	BIGINT PK FK→messages	Üzenet
<code>user_id</code>	INT PK FK→users	Fogadó
<code>delivered_at</code>	DATETIME	Kézbesítés ideje (NULL = nem kézbesítve)
<code>read_at</code>	DATETIME	Olvasás ideje (NULL = nem olvasott)

3. REST API

3.1 Alap URL és hitelesítés

Base URL: `https://babl.rv42.hu:9456/tricc/api`
 Auth: `Authorization: Bearer <jwt_token>`

A JWT token bejelentkezéskor kapott, `HS256` algoritmussal aláírt. A payload tartalmaz `user_id`-t és `exp` lejáratot.

Minden válasz formátuma:


```
{ "ok": true, "data": { ... } }  
// vagy hiba esetén:  
{ "ok": false, "error": "Hibaüzenet" }
```

3.2 Auth végpontok

Metódus	Végpont	Leírás
POST	/auth/register	Regisztráció meghívóköddal
POST	/auth/login	Bejelentkezés, JWT visszaad
GET	/auth/me	Saját profil lekérése
PUT	/auth/profile	Névmódosítás
PUT	/auth/password	Jelszócsere
POST	/upload/avatar	Profilkép feltöltés

3.3 Szobavégpontok

Metódus	Végpont	Leírás
GET	/rooms	Saját szobák listája
POST	/rooms	Szoba létrehozása
GET	/rooms/{id}	Szoba részletei + tagok
POST	/rooms/{id}/members	Tag hozzáadása
DELETE	/rooms/{id}/members/{uid}	Tag eltávolítása
POST	/rooms/{id}/read	Olvasottnak jelölés
POST	/rooms/{id}/hide	Szoba elrejtése
POST	/rooms/{id}/mute	Némítás

DELETE	/rooms/{id}/mute	Némítás feloldása
POST	/rooms/{id}/pin	Üzenet kitűzése
DELETE	/rooms/{id}/pin	Kitűzés törlése
POST	/rooms/{id}/delete-request	Törlési kérés
POST	/rooms/{id}/keep	Törlési kérés visszavonása

3.4 Üzenetvégpontok

Metódus	Végpont	Leírás
GET	/rooms/{id}/messages	Üzenetek lapozva
POST	/rooms/{id}/messages	Üzenet küldése
PUT	/rooms/{id}/messages/{mid}	Üzenet szerkesztése
DELETE	/rooms/{id}/messages/{mid}	Üzenet törlése
POST	/rooms/{id}/messages/{mid}/reactions	Reakció toggle
GET	/rooms/{id}/messages/search	Keresés
GET	/rooms/{id}/media	Média galéria

3.5 Fájlfeltöltés

Metódus	Végpont	Leírás
POST	/upload	Általános fájl / videó feltöltése
POST	/upload/avatar	Profilkép feltöltése

Mindkét végpont `multipart/form-data` formátumot vár, `file` mezőnévvel. Videók esetén a szerver a `video` MIME típust felismeri és `type: "video"` értéket ad vissza.

3.6 Push token végpontok

Metódus	Végpont	Leírás
POST	/push/register	Token regisztrálása
DELETE	/push/register	Token törlése (kijelentkezéskor)

3.7 Hanghívás végpontok

Metódus	Végpont	Leírás
POST	/call/token	LiveKit JWT token generálás
POST	/rooms/{id}/call/notify	Hívás értesítő broadcast indítása

LiveKit token kérés:

```
// Kérés:
{ "room_id": 5 }

// Válasz:
{
  "url": "wss://babl.rv42.hu:7880",
  "token": "eyJ...",
  "room": "room_5"
}
```

A token generálásához a szerver ellenőrzi, hogy a hívó tagja-e a szobának. A JWT `video` claim tartalmaz szoba-specifikus jogosultságot (`roomJoin: true` , szoba neve).

Hívás értesítő (POST /rooms/{id}/call/notify):

Kiváltja a WS szerver által küldött `call_started` broadcastot az adott szoba tagjainak. APNs/FCM push értesítés is küldésre kerül azoknak a tagoknak, akik nem online a WS-en.

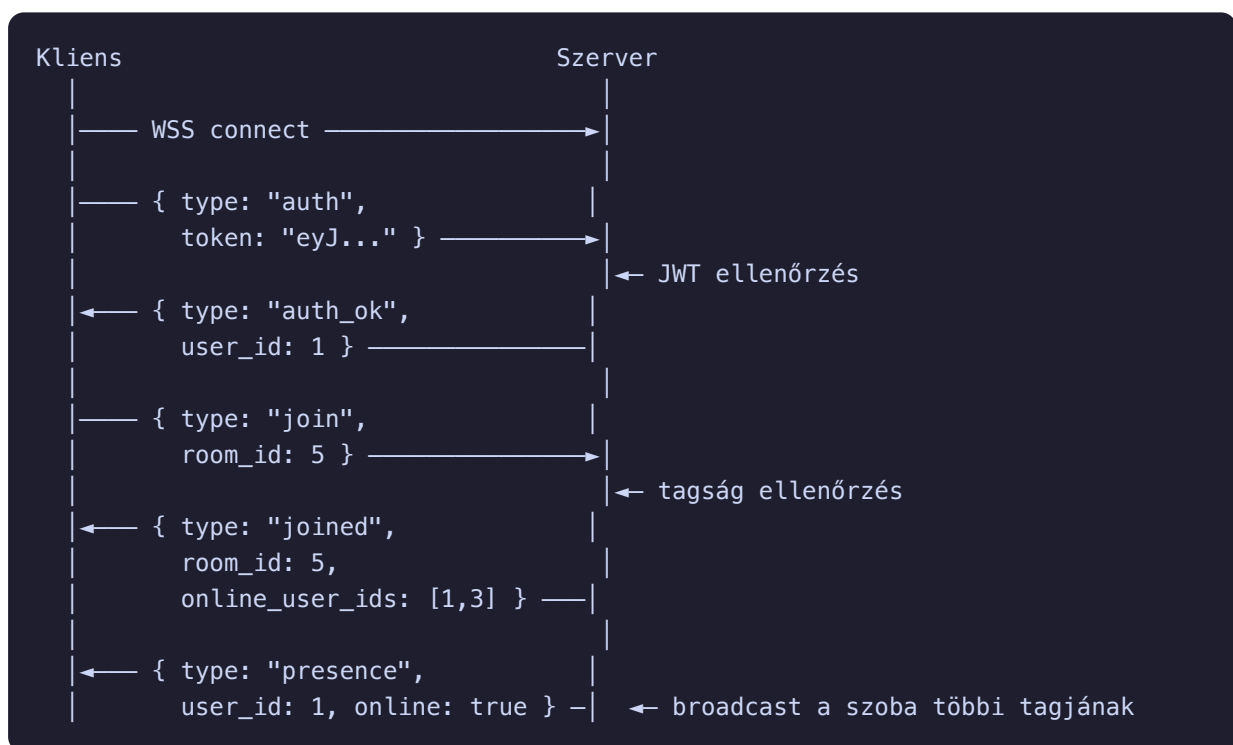
```
// Válasz:
{ "ok": true }
```

3.8 Admin végpontok

Metódus	Végpont	Leírás
GET	/users	Összes felhasználó listája
GET	/admin/invite-codes	Meghívó kódok
POST	/admin/invite-codes	Új meghívó kód generálás
PUT	/admin/users/{id}	Felhasználó módosítása (tiltás, admin)

4. WebSocket protokoll

4.1 Kapcsolat felépítése



4.2 Kliens → Szerver üzenetek

Típus	Leírás	Mezők
-------	--------	-------

<code>auth</code>	JWT hitelesítés	<code>token</code>
<code>join</code>	Szobához csatlakozás	<code>room_id</code>
<code>leave</code>	Szoba elhagyása	<code>room_id</code>
<code>typing</code>	Gépelés jelzés	<code>room_id</code> , <code>typing: bool</code>
<code>delivered</code>	Kézbesítési ACK	<code>message_id</code> , <code>room_id</code>
<code>ping</code>	Heartbeat	—

4.3 Szerver → Kliens üzenetek

Típus	Leírás	Főbb mezők
<code>auth_ok</code>	Sikeres hitelesítés	<code>user_id</code>
<code>joined</code>	Szobába léptünk	<code>room_id</code> , <code>online_user_ids</code>
<code>message</code>	Új üzenet érkezett	<code>room_id</code> , <code>message: {...}</code>
<code>typing</code>	Valaki gépel	<code>room_id</code> , <code>user_id</code> , <code>typing</code>
<code>presence</code>	Online állapotváltozás	<code>user_id</code> , <code>online: bool</code>
<code>presence_list</code>	Aktuális online lista	<code>online_user_ids: [...]</code>
<code>status_update</code>	Kézbesítési státusz frissítés	<code>message_id</code> , <code>user_id</code> , <code>delivered_at</code> , <code>read_at</code>
<code>reaction</code>	Reakció hozzáadva/eltávolítva	<code>message_id</code> , <code>emoji</code> , <code>user_id</code> , <code>action: add/remove</code>
<code>message_edited</code>	Üzenet szerkesztve	<code>room_id</code> , <code>message_id</code> , <code>content</code>
<code>message_deleted</code>	Üzenet törölve	<code>room_id</code> , <code>message_id</code>
<code>member_left</code>	Tag elhagyta a szobát	<code>room_id</code> , <code>user_id</code>
<code>delete_request</code>	Törlési kérés érkezett	<code>room_id</code> , <code>requested_by</code>
<code>user_updated</code>	Profilkép/név változott	<code>user_id</code> , <code>name</code> , <code>avatar_url</code>

<code>call_started</code>	Csoportos hívás indult	<code>room_id</code> , <code>room_name</code> , <code>user_name</code>
<code>call_ended</code>	Csoportos hívás véget ért	<code>room_id</code>
<code>pong</code>	Heartbeat válasz	—
<code>error</code>	Hiba	<code>message</code>

`call_started` esemény részletei

```
{
  "type": "call_started",
  "room_id": 5,
  "room_name": "Fejlesztők",
  "user_name": "Kovács János"
}
```

A WS szerver a `call/notify` REST hívás IPC üzenetére broadcastolja a szoba összes tagjának.

A Flutter kliens `main.dart`-ban fogadja és:

1. `GroupCallService().markRoomCallActive(roomId, roomName)` — állapot frissítés
2. Snackbar megjelenítése "Csatlakozás" gombbal (kivéve ha már ebben a szobában vagyunk hívásban)

`call_ended` esemény részletei

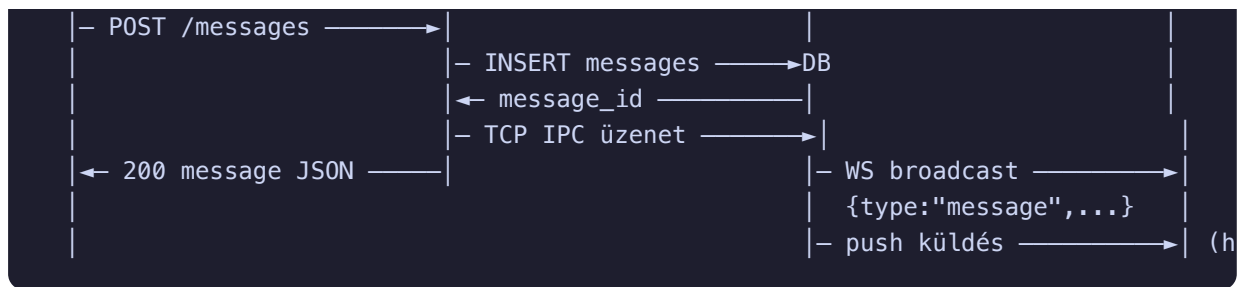
```
{ "type": "call_ended", "room_id": 5 }
```

A szerver a LiveKit webhook `room_finished` eseményére küldi. A kliens:

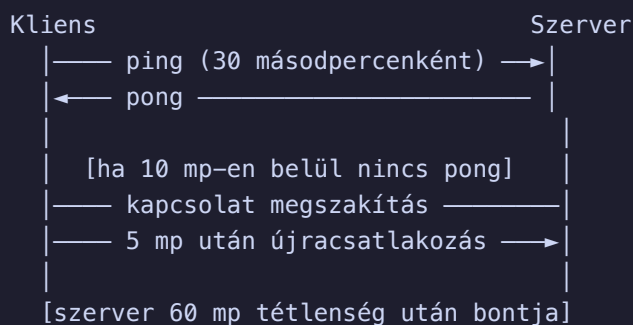
1. `GroupCallService().markRoomCallInactive(roomId)` — állapot törlés
2. Ha aktívan benne volt a hívásban: pip bar eltűnik

4.4 Üzenetküldés folyamat





4.5 Heartbeat mechanizmus



5. Push értesítések

5.1 Mikor küld a szerver push-t?

Push értesítést kap egy felhasználó, ha:

- Nincs aktív WebSocket kapcsolata (offline)
- A szobát **nem némította** el (`is_muted = 0`)
- Kivétel: `@mention` esetén a némítás felülírható (a szerver mindig küld)
- `call_started` esetén a szerver **mindig** küld push-t (hívás értesítő)

5.2 iOS – APNs

Technikai részletek:

- Protokoll: HTTP/2 + JWT token (`.p8` kulcsfájl)
- Key ID: `94HGSV4WAL` , Team ID: `K7Z734X92Z`

- Bundle ID: `com.rv42.babl42`
- Értesítés típus: `alert` (üzenetekhez) és `voip` (1:1 híváshoz, PushKit)

Push payload (üzenet):

```
{
  "aps": {
    "alert": {
      "title": "Kovács János",
      "subtitle": "Fejlesztők",
      "body": "Mikor lesz a deploy?"
    },
    "badge": 3,
    "sound": "default"
  },
  "room_id": 5,
  "message_id": 42
}
```

Push payload (csoportos hívás értesítő):

```
{
  "aps": {
    "alert": {
      "title": "Fejlesztők",
      "body": "Kovács János hanghívást indított"
    },
    "sound": "default"
  },
  "room_id": 5,
  "type": "call_started"
}
```

5.3 Android – FCM

- Firebase Cloud Messaging (FCM)
- `google-services.json` konfiguráció
- Package: `com.rv42.babl42`
- Library: `firebase_messaging: ^15.x`

5.4 Szerver oldali push küldés

```
API kap üzenetet / call_notify kérést
├── WebSocket tagok lekérése (ki online?)
├── Offline tagok → push_tokens lekérése
│   ├── platform = 'ios' → APNs HTTP/2 küldés
│   └── platform = 'android' → FCM küldés
```

6. Flutter kliens architektúra

6.1 Könyvtárstruktúra

```
lib/
├── main.dart                ← App belépési pont, WS eseménykezelő
├── app_theme.dart           ← Light + dark téma (kBlue/kLime)
├── models/
│   ├── message.dart        ← Message, MessageDelivery, ReplyTo, MessageReaction
│   ├── room.dart           ← Room modell
│   └── user.dart           ← User modell
├── services/
│   ├── api_service.dart    ← Összes REST hívás
│   ├── auth_service.dart   ← Token, userId, profil tárolás
│   ├── ws_service.dart     ← WebSocket kapcsolat kezelés
│   ├── push_service.dart   ← APNs (iOS) + FCM (Android)
│   ├── settings_service.dart ← Betűméret, dark mode, SharedPreferences
│   ├── call_service.dart   ← 1:1 WebRTC hívás (CallService)
│   └── group_call_service.dart ← Csoportos LiveKit hívás
├── screens/
│   ├── login_screen.dart
│   ├── register_screen.dart
│   ├── room_list_screen.dart
│   ├── chat_screen.dart
│   ├── profile_screen.dart
│   ├── room_search_screen.dart
│   ├── room_media_screen.dart
│   ├── video_player_screen.dart ← Teljes képernyős videólejátszó
│   └── group_call_screen.dart  ← Csoportos hívás (LiveKit)
```

```
└─ widgets/
   └─ ws_status_bar.dart      ← WsDot, PresenceDot, avatar dialog
   └─ group_call_bar.dart    ← Pip bar (hívás közben állandóan látható)
```

6.2 Service réteg

AuthService (singleton)

- Tárolja: `token`, `userId`, `userName`, `avatarUrl`
- `SharedPreferences` perzisztencia
- `init()` : betöltés induláskor

ApiService (singleton)

- Összes HTTP kérés kezelése
- `Authorization: Bearer <token>` header automatikusan
- Hibakezelés: `ApiException(message, statusCode)`
- Metódusok: `notifyCallStarted(roomId)` — csoportos hívás értesítő

WsService (singleton)

- WebSocket kapcsolat: `wss://babl.rv42.hu:9456/ws`
- Állapotok: `WsState.connected` / `connecting` / `disconnected`
- Automatikus újracsatlakozás 5 másodperc után
- Ping/pong heartbeat: 30s ping, 10s timeout, 60s szerver timeout
- `events` broadcast stream: minden beérkező WS esemény

PushService (singleton)

- iOS: `MethodChannel('push_channel')` → natív AppDelegate (APNs token)
- Android: `FirebaseMessaging` → FCM token
- `_saveAndRegister(token, platform)` : token regisztrálása az API-n

SettingsService (singleton, ChangeNotifier)

- `fontScale` : 0.8–1.5x szövegméret
- `themeMode` : `ThemeMode.system` / `light` / `dark`

GroupCallService (singleton, ChangeNotifier)

A csoportos hanghívások teljes állapotát kezeli. LiveKit SDK (`livekit_client: ^2.4.1`) wrapper.

```
enum AudioOutput { earpiece, speaker, bluetooth }

const _proximityChannel = MethodChannel('com.rv42.babl42/proximity');

class GroupCallService extends ChangeNotifier {
  Room? _room; // LiveKit Room objektum
  bool _connecting = false;
  String? _error;
  int? _chatRoomId; // aktív hívás szoba ID-ja
  String _chatRoomName = '';
  final Map<int, String> _activeCallRooms = {}; // mások által indított hívások
  AudioOutput _audioOutput = AudioOutput.earpiece;
  List<MediaDeviceInfo> _audioDevices = [];
}
```

Főbb metódusok:

Metódus	Leírás
<code>join(roomId, roomName)</code>	LiveKit token kérés → Room.connect → proximity enable → notifyCallStarted
<code>leave()</code>	Room.disconnect → proximity disable → reset → notifyListeners
<code>toggleMute()</code>	LocalParticipant.setMicrophoneEnabled(!isMuted)
<code>reloadAudioDevices()</code>	<code>Helper.audiooutputs</code> → <code>_audioDevices</code> lista frissítés
<code>setAudioOutput(output, {deviceId})</code>	Hangkimenet váltás (lásd lent)
<code>markRoomCallActive(roomId, roomName)</code>	Más szobában lévő hívás regisztrálása
<code>markRoomCallInactive(roomId)</code>	Hívás véget ért, törlés <code>_activeCallRooms</code> -ből
<code>isRoomCallActive(roomId)</code>	<code>true</code> ha van aktív hívás az adott szobában

Hangkimenet váltás:

```
Future<void> setAudioOutput(AudioOutput output, {String? deviceId}) async {
  switch (output) {
    case AudioOutput.earpiece:
      await Helper.setSpeakerphoneOn(false);
    case AudioOutput.speaker:
      await Helper.setSpeakerphoneOn(true);
    case AudioOutput.bluetooth:
      if (deviceId != null) {
        await Helper.selectAudioOutput(deviceId);
      } else {
        await Helper.setSpeakerphoneOnButPreferBluetooth();
      }
  }
  _audioOutput = output;
  notifyListeners();
}
```

Audio eszköz detektálás:

```
Future<void> reloadAudioDevices() async {
  _audioDevices = await Helper.audiooutputs;
  notifyListeners();
}

bool get hasBluetoothDevice => _audioDevices.any((d) {
  final label = d.label.toLowerCase();
  return label.contains('bluetooth') || label.contains('bt') ||
    label.contains('airpod') || label.contains('wireless');
});
```

6.3 App belépési pont (`main.dart`)

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  if (Platform.isAndroid) await Firebase.initializeApp();
  await AuthService().init();
  await SettingsService().init();
}
```

```
runApp(const TriccApp());  
}
```

WS eseménykezelő (`_onWsEvent`):

```
} else if (msg['type'] == 'call_started') {  
    final roomId = msg['room_id'] as int?;  
    final roomName = msg['room_name'] as String? ?? '';  
    final userName = msg['user_name'] as String? ?? 'Valaki';  
    if (roomId != null) svc.markRoomCallActive(roomId, roomName);  
    if (svc.isActive && svc.chatRoomId == roomId) return; // már ebben vagyok  
    // SnackBar: "$userName hanghívást indított – Csatlakozás"  
} else if (msg['type'] == 'call_ended') {  
    final roomId = msg['room_id'] as int?;  
    if (roomId != null) GroupCallService().markRoomCallInactive(roomId);  
}
```

MaterialApp builder — pip bar:

```
builder: (context, child) => MediaQuery(  
    data: MediaQuery.of(context).copyWith(textScaler: ...),  
    child: Column(  
        children: [  
            Expanded(child: child!),  
            GroupCallBar(navigatorKey: navigatorKey),  
        ],  
    ),  
),
```

Miért `navigatorKey` : A `MaterialApp.builder` context az alkalmazás navigátor **felett** van.

Standard `Navigator.push(context, ...)` itt nem működik. Megoldás:

`GlobalKey<NavigatorState> navigatorKey` átadása `GroupCallBar` -nak, és
`navigatorKey.currentState?.push(...)` használata.

7. Képernyők

7.1 Bejelentkezési képernyő (`login_screen.dart`)

Email + jelszó mezők, `POST /auth/login`, token mentés, átirányítás RoomListScreen-re.

7.2 Regisztrációs képernyő (`register_screen.dart`)

Név, email, jelszó, meghívókód mezők, `POST /auth/register`.

7.3 Szoba lista képernyő (`room_list_screen.dart`)

Csoport létrehozás bottom sheet (újítás v1.2.0):

- `isScrollControlled: true` — a sheet a teljes képernyő 85%-át töltheti ki
- `ConstrainedBox(maxHeight: screenHeight * 0.85)` + `Flexible` `ListView`
- Kiválasztott tagok vízszintes chip-sorban jelennek meg (X-szel eltávolítható)

7.4 Chat képernyő (`chat_screen.dart`)

Csoportos hívás gomb (3 állapotú):

```
final isMyCall    = svc.isActive && svc.chatRoomId == _room.id;
final isJoinable  = svc.isRoomCallActive(_room.id) && !isMyCall;
final isBusy      = svc.isActive && svc.chatRoomId != _room.id;

// isMyCall → piros call_end ikon, kilépés a hívásból
// isJoinable → zöld headset ikon, csatlakozás
// isBusy → szürke headset ikon, letiltva (más szobában vagyok hívásban)
// else → normál headset ikon, hívás indítása
```

7.5 Csoportos hívás képernyő (`group_call_screen.dart`)

Konstruktor: `GroupCallScreen({required int roomId, required String roomName})`

Inicializálás: `_svc.join(widget.roomId, widget.roomName)` az `initState`-ben.

Nincs PopScope — a vissza gomb minimalizál, nem hagyja el a hívást:

```

AppBar(
  leading: IconButton(
    icon: const Icon(Icons.keyboard_arrow_down),
    onPressed: () => Navigator.pop(context), // csak minimalizál
  ),
  ...
)

```

BottomBar gombok:

1. **Mikrofon** — toggle mute (`_svc.toggleMute()`)
2. **Hangkimenet** — bottom sheet megnyit (`_showAudioSheet`)
3. **Kilépés** — piros `call_end` ikon → `_leave()` → `_svc.leave()` + `Navigator.pop()`

Audio bottom sheet (`_showAudioSheet`):

```

showModalBottomSheet(
  context: context,
  isScrollControlled: true,
  builder: (_) => Column(children: [
    _AudioOption(
      icon: Icons.hearing,
      label: 'Fülhallgató',
      output: AudioOutput.earpiece,
      selected: svc.audioOutput == AudioOutput.earpiece,
    ),
    _AudioOption(
      icon: Icons.volume_up,
      label: 'Kihangosítás',
      output: AudioOutput.speaker,
      selected: svc.audioOutput == AudioOutput.speaker,
    ),
    _AudioOption(
      icon: Icons.bluetooth_audio,
      label: 'Bluetooth',
      output: AudioOutput.bluetooth,
      enabled: svc.hasBluetoothDevice,
      selected: svc.audioOutput == AudioOutput.bluetooth,
    ),
  ]),
);

```

7.6 Pip bar widget (`group_call_bar.dart`)

```
class GroupCallBar extends StatelessWidget {  
  final GlobalKey<NavigatorState> navigatorKey;  
  const GroupCallBar({super.key, required this.navigatorKey});  
  
  // Látható: svc.isActive || svc.isConnecting  
  // onTap: navigatorKey.currentState?.push(GroupCallScreen route)  
  // Tartalom: headset ikon + szoba neve + résztvevő szám + mic toggle + kilépés  
}
```

7.7 Videólejátszó képernyő (`video_player_screen.dart`)

- `video_player: ^2.9.2` alapú teljes képernyős lejátszó
- Inicializálás URL-ből (`VideoPlayerController.networkUrl`)
- Lejátszás / szünet, előre/hátra tekerés
- HTTPS streamelés Let's Encrypt tanúsítvánnyal (iOS: `AVPlayer` , Android: `ExoPlayer`)

7.8 Profil képernyő (`profile_screen.dart`)

- Profilkép, névmódosítás, betűméret, dark mode, jelszócsere
- **Névjegy (About)** szekció: `package_info_plus` alapján automatikus verzió és build szám megjelenítés

7.9 Keresési képernyő (`room_search_screen.dart`)

- Valós idejű keresés (0.5s debounce)
- Találatra nyomás: `Navigator.pop(context, message)` → ChatScreen megkapja

7.10 Média galéria (`room_media_screen.dart`)

- `GET /rooms/{id}/media` → képek + videók + fájlok listája
 - Képek rácsos elrendezésben, fájlok lista nézetben
 - Videók: bélyegkép + koppintásra `VideoPlayerScreen`
-

8. Modellek

8.1 Message modell

```
class Message {
    final int id;
    final int roomId;
    final int userId;
    final String userName;
    final String? avatarUrl;
    final String type;           // text, image, file, link, video, system
    final String content;
    final bool isEdited;
    final String? fileUrl;
    final String? fileName;
    final int? fileSize;
    final ReplyTo? replyTo;
    final DateTime createdAt;
    final MessageDelivery? delivery;
    final List<MessageReaction> reactions;
}
```

8.2 Room modell

```
class Room {
    final int id;
    final String name;
    final String type;           // direct / group
    final int unreadCount;
    final String? lastMessage;
    final DateTime? lastMessageAt;
    final User? otherUser;       // csak direct szobánál
    final bool isMuted;
    final Message? pinnedMessage;
    final List<User> members;
}
```

9. Témakezelés (Dark mode)

9.1 Megközelítés

Material 3 `ThemeData` alapú, két téma: `buildAppTheme()` (világos) és `buildDarkTheme()` (sötét). A `MaterialApp` `themeMode` paramétere `SettingsService().themeMode` -ra mutat.

9.2 Brandszínek

Konstans	Érték	Használat
<code>kBlue</code>	<code>#1E5BB5</code>	Saját üzenet buborék, AppBar
<code>kLime</code>	<code>#7CC042</code>	Kiemelések, online jelzők
<code>kBlueDark</code>	<code>#1A3A6E</code>	Dark mód AppBar

9.3 3-fokozatú kapcsoló

```
Profil képernyő → _ThemeModeSection
├─ SegmentedButton<ThemeMode>:
│   └─ [🌙 Rendszer] [☀️ Világos] [🌑 Sötét]
│       └─ SettingsService().setThemeMode(mode)
│           └─ SharedPreferences + notifyListeners() → TriccApp rebu
```

10. Online jelenlét (Presence)

10.1 Működési elv

```
Kliens A csatlakozik
├─
```

```

└─ WS auth → WS szerver
    └─ Közös szobák tagjai kapnak:
        { type: "presence", user_id: A, online: true }
└─ Kliens A join(szoba) →
    { type: "presence_list", online_user_ids: [B, C] }

```

10.2 UI megjelenítés

- **WsDot** : zöld/sárga/piros pötty a WS kapcsolat állapotáról (AppBar)
- **PresenceDot** : zöld karika az avatarokon ha a felhasználó online
- **WsService.onlineUsers** : Set — a jelenleg online user ID-k

11. Kézbesítési státusz

Üzenet elküldve (REST POST)

```

└─ message_deliveries sor létrehozva
    └─ Fogadó ONLINE (WS):
        WS kliens fogadja a 'message' eseményt
        → Küld: { type: "delivered", message_id: X }
        → Szerver: UPDATE delivered_at = NOW()
        → Szerver: broadcast { type: "status_update" } küldőnek
    └─ Fogadó OFFLINE (push):
        APNs/FCM push küldése
        → delivered_at = NOW() (push kézbesítési visszaigazolás alapj

```

Fogadó megnyitja a szobát (GET /messages)

```

└─ POST /rooms/{id}/read → UPDATE read_at = NOW()
    └─ broadcast { type: "status_update" } küldőnek

```

12. Fájlfeltöltés és videó folyamat

```

Felhasználó kiválaszt fájlt / videót (FilePicker / ImagePicker)
|
├─ Megerősítési dialog (fájlnév, méret)
|
▼
POST /upload (multipart/form-data)
|
├─ Szerver menti a fájlt, detektálja a MIME típust
|
├─ Visszakap: { url, file_name, mime, size, type }
|               (type = "video" ha video/* MIME)
|
▼
POST /rooms/{id}/messages
{ type: "video", file_url: url, file_name: ..., file_size: ... }
|
├─ WS broadcast → többi kliens kap "video" típusú üzenetet
|                 → bélyegkép + play gomb a buborékban

```

Videó streamelés:

- iOS: `AVPlayer` — HTTPS streamelés Let's Encrypt tanúsítvánnyal
- Android: `ExoPlayer` — ugyanaz
- Korábbi self-signed cert probléma megoldva Let's Encrypt-re váltással

13. Haptikus visszajelzés és animáció

13.1 Hosszú lenyomás az üzenetbuborékon

```

void _handleLongPressStart(LongPressStartDetails _) {
  HapticFeedback.mediumImpact(); // rezgés azonnal
  setState(() => _pressed = true); // animáció indul
}

void _handleLongPressEnd(LongPressEndDetails _) {
  setState(() => _pressed = false);
  widget.onLongPress?.call();      // menü megnyílik
}

```

13.2 Vizuális effekt

```
AnimatedScale(  
  scale: _pressed ? 0.95 : 1.0,  
  duration: const Duration(milliseconds: 120),  
  curve: Curves.easeOut,  
  child: ConstrainedBox(...), // csak a buborék skálázódik, az avatar nem  
)
```

14. Csoportos hanghívás (LiveKit SFU)

14.1 Architektúra

```
Flutter kliens  
├── POST /call/token → API → LiveKit JWT generálás  
│   │ { url: wss://babl.rv42.hu:7880, token, room: "room_5" }  
├── WSS LiveKit szerver (:7880)  
│   │ Mediasoup/SFU – hangfolyamok keverése szerveren  
│   │ (max ~10 résztvevő egyszerre)  
└── STUN/TURN – coturn (:3478)  
    │ P2P ha lehetséges, relay ha NAT mögött
```

14.2 LiveKit token (szerver oldal)

```
// CallController.php  
$roomName = "room_{$roomId}";  
$payload = [  
  'iss' => LIVEKIT_KEY,  
  'sub' => "user_{$userId}",  
  'iat' => time(),  
  'exp' => time() + 3600,  
  'video' => [  
    'room' => $roomName,  
    'roomJoin' => true,
```

```
'canPublish' => true,
'canSubscribe' => true,
],
];
```

Fontos: A `ListParticipants` Twirp API híváshoz (admin panel) szintén szoba-specifikus token kell — általános `roomAdmin` tokennel 401-et ad.

14.3 Csatlakozás folyamat (Flutter)

```
GroupCallScreen initState
|
└─ GroupCallService.join(roomId, roomName)
    |
    └─ POST /call/token → {url, token, room}
        |
        └─ Room.connect(url, token)
            |
            └─ LiveKit SDK kezeli a WebRTC handshake-t
                |
                └─ LocalParticipant.setMicrophoneEnabled(true)
                    |
                    └─ _proximityChannel.invokeMethod('enable')
                        |
                        └─ iOS: UIDevice.isProximityMonitoringEnabled = true
                            |
                            └─ Android: proximityWakeLock.acquire()
                                |
                                └─ ApiService.notifyCallStarted(roomId)
                                    |
                                    └─ POST /rooms/{id}/call/notify
                                        |
                                        └─ WS szerver broadcast: call_started
                                            |
                                            └─ notifyListeners() → GroupCallBar megjelenik
```

14.4 Résztevők figyelése

```
// room.remoteParticipants: UnmodifiableMapView<String, RemoteParticipant>
// participant.isMicrophoneEnabled(): metódus (nem getter)

List<RemoteParticipant> get participants =>
  _room!.remoteParticipants.values.toList();
```

A `Room` extends `DisposableChangeNotifier` — a `GroupCallService` a room változásaira `notifyListeners()` -t hív, a `GroupCallScreen` `ListenableBuilder` -rel figyeli.

14.5 Hívás véget ér

A LiveKit szerver a hívás befejezésekor `room_finished` webhook eseményt küld az API-nak. Az API:

1. WS IPC üzenet → WS szerver broadcast `call_ended` az összes szoba tagnak
2. Flutter kliens: `GroupCallService().markRoomCallInactive(roomId)`
3. Pip bar eltűnik, chat headset ikon visszaáll normál állapotra

15. Platform channel-ek

15.1 Közelségérzékelő (proximity)

Channel neve: `com.rv42.babl42/proximity`

Metódusok: `enable` / `disable`

```
// GroupCallService
const _proximityChannel = MethodChannel('com.rv42.babl42/proximity');

Future<void> join(...) async {
  // ... csatlakozás
  await _proximityChannel.invokeMethod('enable');
}

Future<void> leave() async {
  // ... lecsatlakozás
  await _proximityChannel.invokeMethod('disable');
}
```

iOS implementáció (`AppDelegate.swift`)

```
private var proximityChannel: FlutterMethodChannel?

// didInitializeImplicitFlutterEngine-ben:
if let proxReg = engineBridge.pluginRegistry.registrar(forPlugin: "ProximityPlugin") {
  let proxCh = FlutterMethodChannel(
    name: "com.rv42.babl42/proximity",
```

```

        binaryMessenger: proxReg.messenger()
    )
    proximityChannel = proxCh
    proxCh.setMethodCallHandler { (call, result) in
        DispatchQueue.main.async {
            switch call.method {
            case "enable":
                UIDevice.current.isProximityMonitoringEnabled = true
            case "disable":
                UIDevice.current.isProximityMonitoringEnabled = false
            default: break
            }
            result(nil as Any?)
        }
    }
}

```

Az `UIDevice.isProximityMonitoringEnabled = true` hatására iOS automatikusan elsötétíti a kijelzőt, ha a proximity szenzor lefedett (telefon archoz kerül).

Android implementáció (`MainActivity.kt`)

```

class MainActivity : FlutterActivity() {
    private var proximityWakeLock: PowerManager.WakeLock? = null

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        val pm = getSystemService(Context.POWER_SERVICE) as PowerManager
        @Suppress("DEPRECATION")
        proximityWakeLock = pm.newWakeLock(
            PowerManager.PROXIMITY_SCREEN_OFF_WAKE_LOCK, "BabL42:proximity"
        )
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, "com.rv42.babl42/p
        ).setMethodCallHandler { call, result ->
            when (call.method) {
                "enable" -> { if (proximityWakeLock?.isHeld == false) proximityWakeLo
                "disable" -> { if (proximityWakeLock?.isHeld == true) proximityWakeLo
                else -> result.notImplemented(); return@setMethodCallHandler
            }
            result.success(null)
        }
    }

    override fun onDestroy() {
        if (proximityWakeLock?.isHeld == true) proximityWakeLock?.release()
        super.onDestroy()
    }
}

```



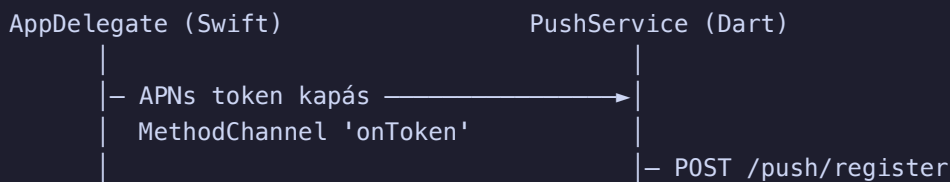
```
}  
}
```

`AndroidManifest.xml` -ben szükséges jogosultság:

```
<uses-permission android:name="android.permission.WAKE_LOCK"/>
```

15.2 APNs token (iOS)

Channel neve: `push_channel`



16. Audio routing (hanghívás)

16.1 flutter_webrtc Helper API

A hangkimenet vezérlése a `flutter_webrtc` csomag `Helper` osztályán keresztül történik:

Metódus	Hatás
<code>Helper.setSpeakerphoneOn(false)</code>	Fülhallgató mód (earpiece)
<code>Helper.setSpeakerphoneOn(true)</code>	Kihangosítás (speakerphone)
<code>Helper.setSpeakerphoneOnButPreferBluetooth()</code>	Bluetooth preferencia, fallback speaker
<code>Helper.selectAudioOutput(deviceId)</code>	Konkrét eszköz kiválasztása
<code>Helper.audiooutputs</code>	<code>Future<List<MediaDeviceInfo>></code> — elérhető eszközök

Megjegyzés: `Helper.enumDevices()` nem létezik — a helyes API: `Helper.audiooutputs` (getter, nem metódus).

16.2 Bluetooth detektálás

```
bool get hasBluetoothDevice => _audioDevices.any((d) {  
  final l = d.label.toLowerCase();  
  return l.contains('bluetooth') || l.contains('bt') ||  
    l.contains('airpod') || l.contains('wireless');  
});
```

Az eszközök listáját `join()` során tölti be automatikusan, majd `setAudioOutput()` hívás előtt is frissíti.

17. iOS specifikus részletek

17.1 APNs integráció (natív Swift)

`AppDelegate.swift` kezeli a token regisztrációt és a proximity channel-t. A `FlutterImplicitEngineDelegate` mintát követi, amely `SceneDelegate` alapú appokhoz szükséges.

17.2 Alkalmazás ikonok

- `flutter_launcher_icons` package
- Forrás: `assets/icon.png`
- iOS: `remove_alpha_channel_ios: true` (App Store követelmény)

17.3 TLS kezelés

- Domain: `babl.rv42.hu` — Let's Encrypt tanúsítvány (korábban önáláírt)
- `Info.plist`: már nincs szükség `NSAppTransportSecurity` exception-re
- iOS videó lejátszás (`AVPlayer`): Let's Encrypt tanúsítvánnyal natívely működik

17.4 Build és kiadás

```
flutter build ios --release --no-codesign

xcodebuild \
  -workspace ios/Runner.xcworkspace \
  -scheme Runner \
  -configuration Release \
  -archivePath ~/Library/Developer/Xcode/Archives/BabL42_vN.xcarchive \
  archive \
  DEVELOPMENT_TEAM="K7Z734X92Z" \
  -allowProvisioningUpdates
```

18. Android specifikus részletek

18.1 FCM integráció

- Package: `firebase_messaging: ^15.x`
- `google-services.json` az `android/app/` mappában
- `com.google.gms:google-services` Gradle plugin

18.2 AndroidManifest.xml főbb elemek

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>

<application android:label="BabL42"
  android:networkSecurityConfig="@xml/network_security_config">
  <!-- FCM Service -->
  <service android:name=".FlutterFirebaseMessagingService" android:exported="fa"
    <intent-filter>
      <action android:name="com.google.firebase.MESSAGING_EVENT"/>
    </intent-filter>
```

```
</service>
</application>
```

18.3 Gradle konfiguráció

```
// android/build.gradle.kts
gradle.afterProject {
    val android = extensions.findByType(com.android.build.api.dsl.LibraryExtension)
    if (android != null && (android.compileSdk ?: 0) < 36) {
        android.compileSdk = 36
    }
}
```

Megjegyzés: A `gradle.afterProject {}` hook azért szükséges, mert egyes plugin-könyvtárak (pl. `file_picker`) alacsonyabb `compileSdk` értéket állítanak be. A hook az összes subproject kiértékelése UTÁN fut le.

18.4 Build

```
flutter build apk --release
# Jövőbeli Play Store: flutter build appbundle
```

19. Biztonsági megfontolások

Terület	Megvalósítás
Hitelesítés	JWT token (HS256), lejáráttal
Jelszó tárolás	bcrypt hash
Regisztráció	Meghívókód szükséges

HTTPS	TLS mindenhol (Let's Encrypt — <code>babl.rv42.hu</code>)
WebSocket	WSS (TLS)
LiveKit	WSS + szoba-specifikus JWT token
Fájlfeltöltés	Szerver oldali típusellenőrzés
Admin jogok	Szerver oldali <code>is_admin</code> ellenőrzés minden admin végponton
Push tokenek	Felhasználóhoz kötve, kijelentkezéskor törlés
Belső hálózat	WireGuard VPN — a szerver csak VPN-en keresztül érhető el

20. Admin panel

Webes felület (`admin/`) az alábbi funkciókkal:

- **Meghívó kód kezelés:** kód generálás, listázás, lejárat beállítása
- **Felhasználó kezelés:** tiltás (`is_active = 0`), admin jog adása/visszavonása
- **Aktív hívások (`calls.php`):** LiveKit API alapú valós idejű megjelenítés

Aktív hívások oldal (`admin/calls.php`)

- LiveKit Twirp API: `ListRooms` → aktív szobák → `ListParticipants` szobánként
- **Token:** szoba-specifikus JWT (`video: {room: "room_N", roomAdmin: true}`)
- 30 másodperces automatikus oldal-frissítés (`<meta http-equiv="refresh">`)
- Megjelenített adatok: szoba neve, résztvevők száma, résztvevők listája

```
function lkApi(string $method, array $payload, string $room = ''): array {
    // room-specifikus JWT generálás + Twirp API hívás a 17880-as porton
    $token = generateLkToken($room);
    // POST https://127.0.0.1:17880/twirp/livekit.RoomService/{$method}
}
```

21. Dependency-k (Flutter)

Package	Verzió	Szerepe
<code>http</code>	^1.2	REST API kérések
<code>web_socket_channel</code>	^3.0	WebSocket kapcsolat
<code>shared_preferences</code>	^2.3	Beállítások perzisztencia
<code>image_picker</code>	^1.1	Kamera / fotótár / videó
<code>file_picker</code>	^8.1	Általános fájlválasztó
<code>cached_network_image</code>	^3.4	Avatar / kép cache
<code>path_provider</code>	^2.1	Fájlrendszer útvonalak
<code>open_filex</code>	^4.5	Letöltött fájl megnyitása
<code>url_launcher</code>	^6.3	URL megnyitása böngészőben
<code>mime</code>	^2.0	MIME típus detektálás
<code>flutter_markdown</code>	^0.7	Markdown renderelés
<code>http_parser</code>	^4.0	MIME típus HTTP fejléc
<code>firebase_core</code>	^3.0	Firebase inicializáció
<code>firebase_messaging</code>	^15.0	Android FCM push
<code>package_info_plus</code>	^9.0	Verzió/build szám lekérdezés
<code>flutter_webrtc</code>	^0.12	WebRTC + audio routing (Helper)
<code>flutter_ringtone_player</code>	^4.0	Csengőhang lejátszás
<code>wake_lock_plus</code>	^1.2	Képernyő bekapcsolva tartás hívás alatt
<code>video_player</code>	^2.9	Videó üzenet lejátszó
<code>livekit_client</code>	^2.4	LiveKit SFU csoportos hanghívás

<code>intl</code>	^0.20	Dátum/idő formázás
<code>flutter_launcher_icons</code>	^0.14	App ikon generálás

22. Verzióhistória

Verzió	Jellemzők
1.0.0	Alapfunkciók: auth, szobák, üzenetek, WebSocket
1.0.5	Fájlküldés, delivery státusz
1.0.8	Emoji reakciók, üzenet szerkesztés, törlés
1.0.9	Média galéria, üzenet keresés
1.0.10	@mention, pin üzenet, szoba elrejtés
1.0.11	Fájlnév fix, push badge, admin fejlesztések
1.1.0	Dark mode (3-fokozatú), keresés→ugrás találatra, jelszócsere, profilkép multi-device szinkron, Android port (FCM push), haptikus visszajelzés + AnimatedScale
1.1.1	Fájl letöltés hosszú nyomás menüből, About modal (package_info_plus), build szám automatikus
1.1.2	1:1 WebRTC hanghívás (CallService), VoIP push (APNs PushKit), hívás képernyő
1.2.0	Videó üzenetek (küldés + VideoPlayerScreen), Let's Encrypt TLS (cert bypass eltávolítva), csoportos hanghívás (LiveKit SFU), pip bar (GroupCallBar, MaterialApp.builder), call_started/call_ended WS esemény + LiveKit webhook, hangkimenet választó (earpiece/speaker/bluetooth, Helper API), közelségérzékelő platform channel (iOS: UIDevice, Android: PROXIMITY_SCREEN_OFF_WAKE_LOCK), admin aktív hívások oldal (LiveKit Twirp API, room-specifikus JWT), csoport létrehozás sheet görgetés javítás + chip sor